# Kadane’s Algorithm – A Competitive Programmer’s Guide (Java)

---

## 1. What is Kadane’s Algorithm?

Kadane’s Algorithm finds the **maximum sum of a contiguous subarray** in **O(n)** time and **O(1)** space.
It’s a classic dynamic programming idea reduced to a greedy choice:

> At each element, should I **extend the previous subarray** or **start a new one**?

This single decision leads to the optimal answer without ever checking every possible subarray.

---

## 2. Problem Statement

Given an integer array `nums`, find the contiguous subarray (containing at least one number) that has the largest sum, and return its sum.

**Example**

```
Input:  [-2, 1, -3, 4, -1, 2, 1, -5, 4]
Output: 6
Explanation: [4, -1, 2, 1] has sum 6.
```

---

## 3. Brute Force & Why We Need Better

Generate every subarray and compute its sum:

- **O($n^3$)** if you sum each subarray naively.
- **O($n^2$)** with a running sum.
- For n = $10^5$ (typical competitive programming limit), O($n^2$) is far too slow.

We must do it in **O(n)**.

---

## 4. The DP Insight (Kadane’s Core)

### 4. The DP Insight (Kadane's Core)

Define **dp[i]** = maximum sum of a subarray **ending at index i**.

Now, for index `i`, you have two choices:

1. **Extend** the best subarray ending at `i-1`: `dp[i-1] + nums[i]`
2. **Start fresh** at `i`: `nums[i]`

So:

```
dp[i] = max(nums[i], dp[i-1] + nums[i])
```

The answer is `max(dp[0], dp[1], ..., dp[n-1])`.

We only need the previous `dp` value → space reduces to **O(1)**.

---

### Why This Works (Prefix-Sum View)

The maximum subarray sum can also be seen as:

```
max_over_j ( prefix[j] - min_{k < j} prefix[k] )
```

Maintaining the minimum prefix seen so far gives O(n) as well.
Kadane's algorithm implicitly does this by tracking the best **current sum** and resetting when it becomes negative.

**Intuition**: A negative prefix will never help the future sum, so discard it.

---

## 5. Algorithm Steps & Dry Run

1. Initialize `currentSum = nums[0]`, `maxSum = nums[0]`.
2. Loop `i` from 1 to n-1:
   - `currentSum = max(nums[i], currentSum + nums[i])`
   - `maxSum = max(maxSum, currentSum)`
3. Return `maxSum`.

**Dry run** on `[4, -1, 2, 1]`:

| i | nums[i] | decision | currentSum | maxSum |
|---|---|---|---|---|
| 0 | 4 | start | 4 | 4 |
| 1 | -1 | max(-1, 4 + (-1)) = 3 | 3 | 4 |

| | | | | |
|---|---|---|---|---|
| 1 | -1 | max(-1, 4+(-1)=3) → 3 | 3 | 4 |
| 2 | 2 | max(2, 3+2=5) → 5 | 5 | 5 |
| 3 | 1 | max(1, 5+1=6) → 6 | 6 | 6 |

Answer: **6**.

---

## 6. Basic Implementation (Java)

```
int maxSubArray(int[] nums) {
    int curr = nums[0], best = nums[0];
    for (int i = 1; i < nums.length; i++) {
        curr = Math.max(nums[i], curr + nums[i]);
        best = Math.max(best, curr);
    }
    return best;
}
```

- Time: **O(n)**
- Space: **O(1)**

---

## 7. Tracking the Subarray Indices

Often you need the actual subarray (start and end) – useful for interviews and output-sensitive problems.

```
int[] maxSubArrayWithIndices(int[] nums) {
    int curr = nums[0], best = nums[0];
    int start = 0, end = 0, tempStart = 0;

    for (int i = 1; i < nums.length; i++) {
        if (nums[i] > curr + nums[i]) {
            curr = nums[i];
            tempStart = i;
        } else {
            curr += nums[i];
        }

        if (curr > best) {
            best = curr;
            start = tempStart;
            end = i;
        }
    }
    return new int[]{best, start, end}; // sum, left, right (inclusive)
```

```
    return new int[]{best, start, end}; // sum, left, right (inclusive)
}
```

For `[-2,1,-3,4,-1,2,1,-5,4]` → returns `[6, 3, 6]` (subarray `[4,-1,2,1]`).

---

## 8. Important Edge Cases

- **All negative numbers**: The algorithm must return the **largest single element** (not 0). Our initialization `curr = best = nums[0]` correctly handles this.
- **Single element**: Loop doesn't run, answer is `nums[0]`.
- **Empty array**: Not allowed by problem constraints (at least one element). If needed, add a check.

---

## 9. Kadane's Template & Variations

Many problems can be solved by **modifying the core recurrence**.
Memorise this template:

```
int curr = arr[0], ans = arr[0];
for (int i = 1; i < n; i++) {
    curr = Math.max(arr[i], curr + arr[i]); // modify this line!
    ans = Math.max(ans, curr);
}
```

### Common Variations

| Variation | Recurrence Change |
|---|---|
| **Max sum subarray** | `max(arr[i], curr + arr[i])` |
| **Min sum subarray** | `min(arr[i], curr + arr[i])` |
| **Max product subarray** | Need `min` and `max` both: `max = max(arr[i], max * arr[i], min * arr[i])` |
| **Maximum circular subarray** | Total sum - min subarray sum (with edge case of all negatives) |
| **Maximum sum with one deletion** | Forward + backward Kadane |
| **Maximum absolute sum** | Max of (max subarray sum, -min subarray sum) |

---

## 10. Important Problems List

### LeetCode

| Difficulty | Problem | Technique |
|---|---|---|
| Easy | 53. Maximum Subarray | Pure Kadane |
| Medium | 918. Maximum Sum Circular Subarray | Kadane + Min Kadane |
| Medium | 152. Maximum Product Subarray | Kadane variant (track min & max) |
| Medium | 1749. Maximum Absolute Sum of Any Subarray | Dual Kadane |
| Medium | 1186. Maximum Subarray Sum with One Deletion | Forward & Backward Kadane |
| Medium | 1191. K-Concatenation Maximum Sum | Kadane on concatenated array |
| Medium | 2272. Substring With Largest Variance | Kadane on frequency difference |
| Hard | 689. Maximum Sum of 3 Non-Overlapping Subarrays | Kadane + DP |
| Hard | 363. Max Sum of Rectangle No Larger Than K | 2D Kadane + Prefix sum + TreeSet |

### GeeksforGeeks

- Maximum Subarray Sum (pure)
- Maximum Circular Subarray Sum
- Maximum Sum Rectangle (2D Kadane)
- Maximum Difference of Zeros and Ones in Binary String (Kadane's on transformed values)

### HackerRank

- **The Maximum Subarray** – requires both contiguous (Kadane) and non-contiguous max sum.
- **Max Array Sum** (House Robber variant, often confused with Kadane – uses similar DP but no contiguity constraint).

---

## 11. 2D Kadane (Maximum Sum Rectangle)

### 11. 2D Kadane (Maximum Sum Rectangle)

Used in problems like "Max Sum of Rectangle No Larger Than K" and "Number of Submatrices That Sum to Target".

**Idea**: Fix left and right columns, compress rows into a 1D array of prefix sums, then run Kadane on that 1D array.
This gives $O(cols^2 \times rows)$ or $O(rows^2 \times cols)$ depending on which side you iterate.

Kadane's is the inner engine of many 2D grid problems.

---

## 12. How to Recognise a Kadane Problem

Look for these keywords or patterns:

- "**contiguous subarray**" + "**maximum**" or "**minimum**" (sum, product, difference, etc.)
- "**circular subarray**"
- "**maximum sum over a continuous range**"
- "**maximum gain / profit / change**" when dealing with consecutive differences.
- The problem can be transformed into a **1D array where you need the best consecutive segment**.

**Red flags** (usually NOT Kadane):

- Non-contiguous elements allowed.
- Subset/subsequence without contiguity.
- Fixed-length window problems (use sliding window instead).

---

## 13. Summary

| Property | Kadane's Algorithm |
|---|---|
| **Purpose** | Maximum sum contiguous subarray |
| **Core recurrence** | `dp[i] = max(nums[i], dp[i-1] + nums[i])` |
| **Time** | O(n) |
| **Space** | O(1) |
| **Key insight** | Discard negative prefixes; they never increase the future sum |
| **Extended uses** | Min subarray, circular max, max product, absolute sum, 2D Kadane, DP |

| | |
|---|---|
| | adaptations |

Kadane's Algorithm is a tiny but powerful tool. Once you understand its **one-line recurrence**, you'll spot it instantly in contests and interviews. Practice the variations listed above, and you'll be able to solve almost any problem that builds on this idea.